Building SPECTRUM

Design Rationale and Implementation of a
Multi-Substrate AI Architecture with Neuromorphic Components

Winston Zai Lin
Blair Academy
linzaiwinston413@gmail.com

April 2026

Abstract

This paper has two goals: to explain *why* the SPECTRUM multi-substrate AI architecture is designed the way it is, and to show *how to build* its core neuromorphic component. The first half derives the SPECTRUM architecture from first principles—the substrate problem, why transformers are the wrong tool for reasoning, and what a physically heterogeneous system looks like. The second half is an implementation guide. We present the DST (Dendritic/Somatic/Threshold) neuron, a three-variable model designed from architectural principles rather than biological replication, and its extension to DSTC (adding a two-compartment dendrite, voltage-gated NMDA coincidence detection, inhibitory synapses, and a calcium plasticity signal). We provide complete Python implementations with annotated equations, working simulations, and the full circuit specification for physical fabrication. The D compartment—an electrolytic capacitor and bleeder resistor—is the first buildable piece; its schematic, component values, and test protocol are given explicitly. SPECTRUM's position is that the ceiling of AI capability is at least partly a substrate problem rather than a purely algorithmic one; this paper does not attempt to establish that, and states it as the motivation for the design rather than as a result. What the paper does establish is narrower and measured: against LIF, Izhikevich and AdEx under matched conditions, DST is the only model tested that holds an absolute refractory period, and its linear dynamics tolerate a coarser integration timestep and lower cost than either nonlinear competitor—which is also what makes the D compartment realisable as a capacitor and a resistor.

1 Why SPECTRUM Exists: The Substrate Problem

Every major AI system today runs on the same substrate: digital silicon executing floating-point matrix multiplication. This is not a principled design choice. It is an accident of history. The transformer architecture [1] achieved dominance because it maps cleanly onto GPU hardware—attention is a large matrix multiply, and GPUs are optimized for exactly that operation.

The limitation is invisible until you look for it. Consider what intelligence actually needs to do:

- **Detect events** in a stream of sensory input—spikes arriving milliseconds apart.
- **Retrieve memories** from partial or corrupted cues—pattern completion under noise.
- **Reason continuously**—run differential equations, simulate physical dynamics, optimize over smooth spaces.
- **Search** large hypothesis spaces—combinatorial optimization.
- **Store** long-term knowledge—persist information over days, months, years.
- **Communicate**—translate internal reasoning into human language.

A digital transformer does all six with the same operation: matrix multiplication over 32-bit floats. It does some of them tolerably. It does none of them optimally. Each of those six functions has a physical substrate for which it is the natural computation:

Function	**Optimal substrate**
Event detection	Neuromorphic (spike-triggered, not clock-triggered)
Pattern completion	Energy-based / Hopfield networks
Continuous reasoning	Analog circuits (differential equations in physics)
Search	Quantum (Grover's $\sqrt{N}$ speedup)
Long-term storage	Molecular (DNA, near-zero passive energy)
Language output	Digital transformer

SPECTRUM's central claim: build a system that routes each function to its physically appropriate substrate. The transformer becomes the output interface—the mouth—not the brain.

1.1 The Energy Argument

In deep-reasoning mode (an autonomous agent running K internal cycles before producing one language output), the energy comparison is stark. A transformer-only system runs K full forward passes at ∼10W each. SPECTRUM runs K cycles on analog (∼5mW) and Hopfield (∼50mW) substrates, invoking the transformer only once at the end:

$$E_{\text{transformer}} = 10\,\text{W} \cdot K, \qquad E_{\text{SPECTRUM}} \approx 55\,\text{mW} \cdot K + 10\,\text{W}$$

The asymptotic ratio is $\approx 180\times$ at current analog hardware, scaling to 10^3–$10^5\times$ for mature analog substrates. The advantage does not exist for single-query chat; it is decisive for long-horizon autonomous reasoning.

2 The SPECTRUM Architecture

SPECTRUM (Synaptic Parallel Ecosystem with Causal Temporal Reasoning Unit Mesh) defines six layers, each on a physically distinct substrate:

Layer	Substrate	Time scale	Power	Function
L1: Perception	Neuromorphic (DSTC)	μs–ms	<10mW	Event detection, coincidence
L2: Working Memory	Hopfield networks	ms–s	∼50mW	Pattern completion, recall
L3: Long Memory	Molecular (DNA)	s–years	∼0	Persistent knowledge
L4: Reasoning	Analog	s–min	∼5mW	Continuous optimization
L5: Search	Quantum	ms–s	variable	Hypothesis space search
L6: Language	Digital (ARBOR)	s	∼10W	Output, routing, synthesis

Table 1: SPECTRUM layer specification. All layers communicate through a shared 1024-dimensional continuous embedding space. The only cross-layer coupling is this embedding; each substrate is otherwise physically independent.

Three design principles govern every choice:

1. **Language is the output, not the reasoning.** L6 is the interface. The actual computation happens in L1–L5.
2. **Time scales are explicit.** Each layer operates at its natural timescale. Nothing is forced to a single clock.
3. **Memory and compute are substrate-separated.** L3 stores; L2 retrieves; L4 reasons. No layer does all three.

Information flow. Input enters at L1 (fast perceptual processing). Salient patterns pass to L2 (working memory). L2 retrieves relevant context from L3. The assembled context enters L4 (reasoning), which may invoke L5 (search) for optimization subproblems. L6 translates the result into language. Critically, L4 can loop back to L1, L2, and L3 internally without producing a language output—internal reasoning is substrate-efficient.

3 The DST Neuron: Design from Scratch

L1 requires a neuromorphic unit: a circuit that fires spikes in response to input, integrates inputs over time, and resets after firing. The standard approach borrows from biology—the Hodgkin-Huxley (HH) model [2], with four coupled variables and six ionic conductance functions calibrated to squid axon data. HH is accurate and biologically faithful. It is also the wrong starting point for a chip.

We derive the DST neuron differently: from the three computational jobs a neuron has to do.

3.1 Three Jobs, Three Variables

$D(t)$ — **Dendritic potential.** The dendrite integrates incoming synaptic currents over a time window. Inputs that arrive close together should add up; inputs spread far apart should not. This is leaky integration:

$$\tau_D \frac{dD}{dt} = -D + I_{\text{syn}}(t)$$

D charges toward I_{syn} and leaks back to zero with time constant $\tau_D = 10\,\text{ms}$.

$S(t)$ — **Somatic potential.** The soma decides whether accumulated dendritic drive is sufficient to fire. It sits at rest ($S_{\text{rest}} = -1$) and is pushed toward threshold by D, resisted by the adaptive threshold θ:

$$\tau_S \frac{dS}{dt} = -(S - S_{\text{rest}}) + \kappa D - (\theta - \theta_0)$$

Fire condition: $S(t) \geq 0$. After firing: $S \leftarrow S_{\text{reset}} = -2$.

$\theta(t)$ — **Adaptive threshold.** After each spike, θ jumps by $\Delta\theta = 6$, making it temporarily impossible to fire again (absolute refractory). θ then decays back to baseline $\theta_0 = 1$ with time constant $\tau_\theta = 80\,\text{ms}$ (relative refractory):

$$\frac{d\theta}{dt} = -\frac{\theta - \theta_0}{\tau_\theta}, \quad \theta(t_k^+) = \theta(t_k^-) + \Delta\theta$$

Together these three equations produce: single action potentials, temporal summation, spatial summation, absolute and relative refractory periods, and spike-rate adaptation—the full behavioral repertoire of a spiking neuron, with three variables instead of HH's four plus six conductance functions.

3.2 Analytical Spike Waveform

HH simulates the spike shape by integrating ion channel dynamics at every timestep. DST takes a different approach: the spike waveform is *analytical*. At each firing time t_k, the displayed voltage is:

$$V(t) = V_{\text{rest}} + A_{\text{peak}} \cdot \mathcal{G}(t - t_k,\, \sigma_{\text{rise}}) - A_{\text{AHP}} \cdot \mathcal{G}(t - t_k - \tau_{\text{AHP}},\, \sigma_{\text{AHP}})$$

where $\mathcal{G}(x, \sigma) = \exp(-x^2/2\sigma^2)$. Two Gaussians: one for the depolarization peak ($A_{\text{peak}} = 100\,\text{mV}$, $\sigma_{\text{rise}} = 0.25\,\text{ms}$), one for the afterhyperpolarization ($A_{\text{AHP}} = 12\,\text{mV}$, $\tau_{\text{AHP}} = 1.2\,\text{ms}$ offset, $\sigma_{\text{AHP}} = 2\,\text{ms}$). This waveform is computed in $O(N_{\text{fires}})$ rather than being integrated at every timestep. It is $\sim$10$\times$ faster than HH and produces a cleaner, tunable spike shape.

3.3 Parameters

Parameter	Value	Meaning
τ_D	10 ms	Dendritic integration window (EPSP decay)
τ_S	2 ms	Somatic response speed
τ_θ	80 ms	Threshold recovery (refractory timescale)
κ	1.2	Dendritic-to-somatic coupling gain
S_{rest}	-1.0	Somatic resting level
θ_0	1.0	Baseline threshold
$\Delta\theta$	6.0	Threshold jump per spike
S_{reset}	-2.0	Somatic reset after fire

4 The DSTC Extension: Four Additions

DST is minimal and fast. DSTC adds four mechanisms that make the chip simulate real neurons more faithfully—and, more importantly, give L1 computational capabilities beyond simple threshold integration.

4.1 Two-Compartment Dendrite

Real dendrites are not a single point. Distal inputs (far from the soma) integrate more slowly than proximal inputs (near the soma). We split D into two compartments in cascade:

$$\tau_{D_d}\frac{dD_d}{dt} = -D_d + I_{\text{AMPA}}(t) + g_{\text{NMDA}}(D_d)\, I_{\text{NMDA}}(t) - I_{\text{inh},d}(t) \tag{1}$$

$$\tau_{D_p}\frac{dD_p}{dt} = -D_p + I_{\text{prox}}(t) + \kappa_{dd}\, D_d - I_{\text{inh},p}(t) \tag{2}$$

with $\tau_{D_d} = 15\,\text{ms}$ (distal, slower) and $\tau_{D_p} = 5\,\text{ms}$ (proximal, faster). Distal signals travel through $D_d \to D_p$ before reaching S, introducing a natural propagation delay. Proximal inhibition ($I_{\text{inh},p}$) is more effective than distal inhibition ($I_{\text{inh},d}$) because it intercepts the signal closer to the soma—consistent with known shunting inhibition biology.

4.2 NMDA-Like Voltage Gate

The NMDA receptor is biologically unique: it requires both a presynaptic spike *and* postsynaptic depolarization to open. It is a coincidence detector. We implement the Mg^{2+} block relief as:

$$g_{\text{NMDA}}(V) = \frac{1}{1 + \gamma \cdot e^{-\beta V}}, \quad \gamma = 3.0,\ \beta = 2.5$$

At rest ($V_{D_d} = -1$): $g_{\text{NMDA}} \approx 0.027$—nearly blocked. At depolarization ($V_{D_d} = 2$): $g_{\text{NMDA}} \approx 0.98$—fully open. An NMDA-only input arriving at a resting dendrite produces almost no effect. The same input arriving when the dendrite is already depolarized (e.g., by

concurrent AMPA input) fires the neuron. This is hardware-level coincidence detection: L1 detects correlated inputs without digital computation.

Hardware implementation: a MOSFET whose gate voltage is V_{D_d}. The source-drain conductance varies with gate voltage exactly as $g_{\text{NMDA}}(V)$ varies with V_{D_d}.

4.3 Inhibitory Synapses

Inhibitory inputs pull the dendritic potential toward a reversal potential $E_{\text{inh}} = -2.5$ (below S_{rest}):

$$I_{\text{inh}}(t) = g_{\text{inh}}(t) \cdot (V - E_{\text{inh}})$$

This is conductance-based (shunting) inhibition: the inhibitory effect is strongest when the membrane is depolarized and weakest at rest—mirroring biological GABA_A receptor dynamics. Hardware: NPN transistor current sink.

4.4 Calcium Compartment and STDP

A calcium compartment $Ca(t)$ rises by $\delta_{\text{Ca}} = 1$ at each spike and decays with $\tau_{\text{Ca}} = 200\,\text{ms}$:

$$\frac{dCa}{dt} = -\frac{Ca}{\tau_{\text{Ca}}}, \quad Ca(t_k^+) = Ca(t_k^-) + \delta_{\text{Ca}}$$

Ca tracks recent spike history. It is the plasticity signal for spike-timing-dependent plasticity (STDP): the weight of a synapse increases if the presynaptic spike arrives before the postsynaptic spike (LTP) and decreases otherwise (LTD), with an exponential time window:

$$\Delta w = \begin{cases} A_+ \cdot e^{-\Delta t/\tau_+} & \Delta t > 0 \text{ (LTP)} \\ -A_- \cdot e^{\Delta t/\tau_-} & \Delta t < 0 \text{ (LTD)} \end{cases}$$

This enables L1 to *learn online*—adapting synaptic weights during inference without invoking the digital transformer at L6. Hardware: a slow RC circuit ($200\,\mu\text{F} \times 1\,\text{k}\Omega$, $\tau = 200\,\text{ms}$) charged by the spike output of the S compartment.

4.5 Full DSTC System

The complete five-variable system is:

$$\begin{aligned}
\tau_{D_d}\dot{D}_d &= -D_d + I_{\text{AMPA}} + g_{\text{NMDA}}(D_d) I_{\text{NMDA}} - g_{\text{inh},d}(D_d - E_{\text{inh}}) \\
\tau_{D_p}\dot{D}_p &= -D_p + I_{\text{prox}} + \kappa_{dd} D_d - g_{\text{inh},p}(D_p - E_{\text{inh}}) \\
\tau_S \dot{S} &= -(S - S_{\text{rest}}) + \kappa D_p - (\theta - \theta_0) \\
\dot{\theta} &= -(\theta - \theta_0)/\tau_\theta \\
\dot{Ca} &= -Ca/\tau_{Ca}
\end{aligned}$$

with spike events at $S \geq 0$: $S \leftarrow S_{\text{reset}}$, $\theta \leftarrow \theta + \Delta\theta$, $Ca \leftarrow Ca + \delta_{Ca}$.

5 Python Implementation

The full implementation lives in three files. We present the key functions here.

5.1 `dst_model.py`: The Base Simulator

The core simulator integrates the three DST equations with Euler's method. Synaptic inputs are precomputed as exponentially-decaying currents from a list of spike times:

```python
def simulate(I_func, synapses=None, t_end=100, dt=0.01):
    t  = np.arange(0, t_end, dt)
    D  = np.zeros(len(t))
    S  = np.zeros(len(t)); S[0] = S_rest
    TH = np.full(len(t), th_0)
    fires = []

    # precompute synaptic input at each timestep
    I_syn = np.zeros(len(t))
    for i, ti in enumerate(t):
        I_syn[i] = I_func(ti)
        if synapses:
            for w, spike_times in synapses:
                for ts in spike_times:
                    if ts <= ti:
                        I_syn[i] += w * np.exp(-(ti - ts) / tau_D)

    for i in range(1, len(t)):
        d = D[i-1]; s = S[i-1]; th = TH[i-1]
        dD  = (-d  + I_syn[i]) / tau_D
        dS  = (-(s - S_rest) + kappa * d - (th - th_0)) / tau_S
        dth = -(th - th_0) / tau_th
        D[i]  = d  + dD  * dt
        S[i]  = s  + dS  * dt
        TH[i] = th + dth * dt
        if S[i] >= 0:          # fire condition
            fires.append(t[i])
            S[i]  = S_reset
            TH[i] = th + dth * dt + dth   # theta jumps
    V = spike_waveform(t, fires)
    return t, D, S, TH, V, fires
```

Listing 1: DST simulator core (`dst_model.py`)

5.2 `dstc_model.py`: The NMDA Gate and Calcium

The NMDA gate is a single line. The full DSTC simulator adds the distal compartment, inhibition, and calcium:

```python
def g_NMDA(V):
    """Voltage-gated NMDA conductance.
    Nearly 0 at rest (V=-1), ~1 when depolarized (V=2)."""
    return 1.0 / (1.0 + gamma_N * np.exp(-beta_N * V))
```

```
5
6  # Inside the integration loop:
7  nmda_in = g_NMDA(dd) * I_nmda_d(t[i])
8  inh_d   = I_inh_d(t[i]) * (dd - E_inh)   # shunting inhibition
9  inh_p   = I_inh_p(t[i]) * (dp - E_inh)
10
11 dDd = (-dd + I_syn_d[i] + nmda_in - inh_d) / tau_Dd
12 dDp = (-dp + I_syn_p[i] + kappa_dd * dd - inh_p) / tau_Dp
13 dS  = (-(s - S_rest) + kappa * dp - (th - th_0)) / tau_S
14 dth = -(th - th_0) / tau_th
15 dCa = -ca / tau_Ca
16
17 # ... update state ...
18
19 if S[i] >= 0:           # fire
20     fires.append(t[i])
21     S[i]  = S_reset
22     TH[i] = th + dth * dt + dth
23     Ca[i] = ca + dCa * dt + delta_Ca   # Ca jump
```

Listing 2: NMDA gate and DSTC step (`dstc_model.py`)

5.3 `D_compartment.py`: The Physical Circuit Simulator

The D compartment is a capacitor and resistor. The simulator integrates Kirchhoff's current law directly, making it a direct correspondence between equation and component:

```
1  # Circuit: tau_D = R_bleed * C_cap = 1kOhm * 10uF = 10ms
2  C_cap   = 10e-6    # 10 uF capacitor
3  R_bleed = 1e3      # 1 kOhm bleeder resistor (the leak)
4  R_syn   = 2e3      # 2 kOhm per synapse (weight = 1/R_syn)
5
6  def simulate_RC(input_sources, t_end=0.08, dt=0.0001):
7      t = np.arange(0, t_end, dt)
8      V = np.zeros(len(t))   # voltage across capacitor
9      for i in range(1, len(t)):
10         v    = V[i-1]
11         I_leak = v / R_bleed      # current through bleeder (leak)
12         I_in   = 0.0
13         for amp, ts, te, rw in input_sources:
14             if ts <= t[i-1] < te:
15                 I_in += (amp - v) / rw   # current through weight
                       resistor
16         dV   = (I_in - I_leak) / C_cap
17         V[i] = v + dV * dt
18     return t * 1000, V    # return time in ms
```

Listing 3: RC circuit simulator (`D_compartment.py`)

The equation $\tau_D \dot{V} = -V + \sum_i w_i I_i(t)$ is exactly what this circuit computes: `C_cap` is C, `R_bleed` is R (so $\tau_D = RC$), and `R_syn` sets the synaptic weight ($w_i = 1/R_w$).

6 Simulation Results

Each model file produces an 8-panel figure. We describe the key demonstrations.

6.1 DST Behavioral Repertoire (`dst_model.py`)

Single spike (Panel 1). A 2 ms current injection at $t = 10$ ms produces a clean action potential: D charges, S crosses threshold, θ jumps, S resets below rest, θ recovers over $\sim$80 ms.

Refractory (Panel 3). A second identical stimulus 4 ms after the first is completely blocked (θ is still elevated). A third stimulus at $t = 28$ ms fires successfully once θ has recovered.

Temporal summation (Panel 4). A single sub-threshold pulse fails to fire. Four identical pulses spaced 4 ms apart (within the $\tau_D = 10$ ms integration window) accumulate in D and fire the neuron. The same pulses spaced 10 ms apart do not.

Spatial summation (Panel 5). One synapse of weight 0.5 is sub-threshold. Three simultaneous synapses of the same weight fire the neuron. Six synapses produce a rapid burst.

Spike-rate adaptation (Panel 6). Under sustained current injection, each spike raises θ, so the next spike requires θ to recover further before S can reach threshold again. The inter-spike interval grows monotonically: the neuron adapts its firing rate to sustained input, a hallmark of biological neurons.

6.2 DSTC Extensions (`dstc_model.py`)

NMDA gate (Panel 2). $g_{\text{NMDA}}(V)$ is plotted across $V \in [-3, 4]$. At $V = -1$ (rest): $g = 0.027$. At $V = 0$ (threshold): $g = 0.25$. At $V = 2$ (depolarized): $g = 0.98$. The gate traverses three orders of magnitude across the physiological operating range.

Coincidence detection (Panel 3). NMDA input alone (g_{NMDA} blocks it at rest) does not fire the neuron even at large amplitude. The same NMDA input combined with concurrent AMPA input (which depolarizes D_d, opening the gate) fires a burst of spikes. This demonstrates two-input coincidence detection in hardware.

Distal-to-proximal delay (Panel 4). A distal input drives D_d (peak at $\sim$15 ms), which then charges D_p (peak delayed by several ms). The propagation delay through the dendritic cable is visible directly.

Inhibitory shunting (Panel 5). Proximal inhibition (blocking D_p) is more effective than distal inhibition (blocking D_d) at the same conductance. A stimulus that fires the neuron without inhibition is blocked by proximal inhibition and only weakened by distal inhibition.

Calcium accumulation (Panel 6). Under sustained input, Ca builds up across spikes and decays slowly ($\tau_{Ca} = 200$ ms). This staircasing calcium trace is the plasticity signal: a synapse consistently active before spikes will see rising Ca and undergo LTP.

6.3 D Compartment, Simulated Circuit Response (`D_compartment.py`)

The figures in this subsection are produced by a circuit simulator, not measured on hardware. The component values are those specified in Section 7; the traces are what those values predict.

Circuit (Panel 1). The schematic: three synaptic inputs each through a 2 kΩ weight resistor, converging on a 10 μF capacitor with a 1 kΩ bleeder to ground. Output is the capacitor voltage V_D.

τ verification (Panel 2). A single 5 ms input pulse charges V_D. The decay from peak is measured; the time to reach $V_{\text{peak}} \cdot e^{-1}$ is $\tau_D = 10$ ms, confirming $RC = 10$ ms.

Spatial summation (Panel 3). One synapse peaks at ∼0.22 V. Two simultaneous synapses at ∼0.39 V. Three at ∼0.52 V. The D-to-S handoff threshold is set at 0.45 V: one synapse sub-threshold, two borderline, three fires.

Temporal summation (Panel 4). Two pulses 8 ms apart (within τ_D) stack: the second arrives before the first has decayed, accumulating on the residual charge. Two pulses 45 ms apart fully decay between them.

7 Hardware Build Path

7.1 D Compartment (Specified; Build Pending)

Component	Value	Function
Electrolytic capacitor C	10 μF	Integrates charge (the memory)
Bleeder resistor R	1 kΩ	Leak to ground ($\tau_D = RC = 10$ ms)
Synaptic resistors R_w	1–10 kΩ	Weight: $w_i = 1/R_{w,i}$

The bench for this measurement is implemented in `bench/` (firmware, capture, and an overlay that fits τ_D from the recorded trace and reports the capacitance it implies). No measured data is reported in this paper; the protocol below is what the bench executes.

Test protocol. (1) Apply a 2 V pulse for 5 ms through one synaptic resistor. Measure V_D with an oscilloscope. Confirm exponential decay with $\tau = 10$ ms. (2) Apply two simultaneous pulses; confirm V_D peaks at $\sim 2\times$ the single-pulse value. (3) Apply two pulses 8 ms apart; confirm V_D from the second pulse builds on residual from the first. These three tests validate spatial summation, temporal summation, and the τ_D time constant.

7.2 S Compartment (Next)

The soma is a bistable threshold element with self-reset. Hardware: a Schmitt trigger. When V_D crosses the upper threshold, the output switches high (spike); an RC feedback path charges a capacitor that drives the output back low after ∼1 ms (reset). The adaptive threshold θ is an additional slow RC (80 μF × 1 kΩ, $\tau = 80$ ms) charged by the spike output.

7.3 DSTC Full Circuit

Compartment	Hardware	τ	Status
D_d (distal)	$15\,\mu$F $\times$ 1 kΩ RC + MOSFET gate	15 ms	Designed
D_p (proximal)	$5\,\mu$F $\times$ 1 kΩ RC	5 ms	Designed
S (soma)	Schmitt trigger + reset RC	1 ms	Next
θ (threshold)	$80\,\mu$F $\times$ 1 kΩ slow RC	80 ms	Next
Ca (calcium)	$200\,\mu$F $\times$ 1 kΩ slow RC	200 ms	Planned

The MOSFET gate for the NMDA nonlinearity uses the distal dendritic voltage V_{D_d} as the gate control. At rest ($V_{D_d} \approx 0\,$V on the physical scale), the MOSFET is nearly pinched off (conductance low). As V_{D_d} rises (AMPA depolarization), the gate opens and the NMDA current path is enabled. The physical sigmoid matches $g_{\text{NMDA}}(V)$ in the simulation.

8 Integration into SPECTRUM

8.1 DSTC as L1

A network of DSTC neurons instantiates SPECTRUM's L1 (Perception layer). Each neuron exposes three signals to the common 1024-dimensional embedding space:

1. **Spike rate**: spike count over a short integration window (rate-coded scalar).
2. **Calcium level**: Ca voltage, reflecting recent spike history and plasticity state.
3. **Inhibitory control**: a scalar from L2 (working memory) that gates $g_{\text{inh},p}$, implementing top-down attention from the Hopfield layer.

The NMDA coincidence detection means L1 is sensitive to *correlated* inputs, not just strong inputs. The STDP via calcium means L1 learns online without backpropagation. Both properties are absent from existing clock-driven digital neuromorphic chips (Intel Loihi, IBM NorthPole), which implement simplified leaky integrate-and-fire (LIF) neurons without dendritic structure.

8.2 ARBOR as L6

L6 (Language layer) is instantiated as the ARBOR architecture [12]: a **Big Brain** generalist model handles routing and synthesis; a registry of **Long Brain** specialists (each trained exclusively on one domain) provides depth. The Big Brain receives the embedding from L4, identifies required domains, routes to Long Brains, and synthesizes responses.

ARBOR and SPECTRUM are orthogonal: SPECTRUM defines *what physical substrates* compute each function; ARBOR defines *how domain knowledge is organized* at the output stage. A SPECTRUM system reasoning about quantum chemistry uses L5 for hypothesis search, L4 for continuous optimization, and the chemistry Long Brain at L6 for domain-precise output. The Big Brain synthesizes across all three.

8.3 FLUID as Dynamic Topology

The FLUID architecture [13], which modifies neural topology via Hebbian co-activation during inference, runs within each ARBOR Long Brain. This provides morphological computation at the software level inside L6—topology-as-computation without requiring hardware-level reconfigurable photonic meshes.

9 Related Work

Hodgkin-Huxley [2] is the gold standard biological neuron model. DST deliberately departs from it: HH is calibrated to squid axon data; DST is designed for computation. HH requires four variables and six conductance functions; DST requires three variables and no ion channel fits.

Leaky integrate-and-fire (LIF) [6] is the industry standard for neuromorphic chips. LIF has one variable; it cannot produce spike-rate adaptation, NMDA coincidence detection, or dendritic integration. DST adds these at the cost of two additional variables.

Izhikevich [3, 4] is the closest prior work and sets the bar DST must clear. It reaches spike-rate adaptation and a wide firing repertoire with *two* state variables and four parameters, and it was written for the reason given in Section 3—that Hodgkin-Huxley is too expensive to put on a chip. We state plainly that DST does not win on economy: three variables and eight parameters is more, not less. Table 2 reports what DST does buy for that cost.

Adaptive exponential integrate-and-fire (AdEx) [5] likewise obtains adaptation with two variables, via an exponential spike-initiation term and a coupled adaptation current.

Table 2: DST against published spiking-neuron models. All models integrated at $dt = 0.01$ ms over 300 ms; each driven at multiples of its *own* rheobase, since the four carry different current units. Behaviours are read from each model's output rather than asserted. "Max stable dt" is the coarsest timestep that still reproduces the spike count obtained at $dt = 0.01$ ms. Runtime is wall-clock for one 300 ms trace, same host, same dt. Reproduced by `bench/baselines.py`.

Model	Vars	Params	Refractory	Temporal sum.	Adaptation	Max dt (ms)	Runtime
DST (ours)	3	8	**yes**	yes	yes	**2.0**	**12.7**
LIF	1	5	no	yes	no	1.0	6.8
Izhikevich	2	4	no	yes	yes	1.0	23.2
AdEx	2	9	no	yes	yes	0.5	68.1

Two results in Table 2 bear on the design. First, DST is the only model tested that holds an *absolute* refractory period: under a pulse pair at 20× rheobase the others recover through a slow current and can be driven to fire again immediately, whereas the threshold jump $\Delta\theta$ is a hard veto. Second, and more consequential for fabrication, DST tolerates a timestep twice Izhikevich's and four times AdEx's, and at matched dt runs 1.8× faster than Izhikevich and 5.4× faster than AdEx despite the extra state variable. Both competitors pay

for a nonlinearity—a quadratic term requiring sub-stepping, an exponential—while DST's dynamics are entirely linear.

That linearity is the argument, not the variable count. A linear system maps onto an RC network, which is why the D compartment in Section 7 is a capacitor and a resistor; neither a quadratic nor an exponential term has any comparable two-component realisation. The point-neuron comparison also cannot score the DSTC extensions at all, since a point neuron has no morphology on which to place a two-compartment dendrite, a voltage-gated NMDA conductance, or a proximal/distal inhibition asymmetry.

Intel Loihi [7] and **IBM NorthPole** [8] are state-of-the-art neuromorphic chips using LIF neurons. Neither integrates with transformer language models. SPECTRUM proposes this integration with DSTC at L1.

Multi-compartment models [9] extend HH with spatial dendritic structure. DSTC takes a minimal two-compartment approach (distal + proximal) sufficient to demonstrate NMDA coincidence detection and shunting inhibition without the parameter burden of full compartmental models.

STDP [10] is well-established experimentally. The calcium-based STDP in DSTC follows the framework of [11], using Ca as the plasticity induction variable rather than directly tracking spike timing.

10 Conclusion

The argument for SPECTRUM is simple: intelligence needs different physical machinery for different cognitive functions, and building it all on one substrate is leaving capability and energy efficiency on the table. The transformer is not wrong; it is mispositioned. It belongs at the output, not the core.

The DST neuron shows that a neuromorphic unit can be derived from first principles—three equations, three variables, eight parameters—and produce the behavioral repertoire of a spiking neuron without biological fitting. That repertoire alone is not the contribution: Izhikevich reaches it with two variables and four parameters [3]. What DST adds, per Table 2, is an absolute refractory period the two-variable models lack, and fully linear dynamics—which cost less to integrate and, unlike a quadratic or exponential term, correspond directly to passive components. DSTC extends this with NMDA coincidence detection, two-compartment dendritic integration, inhibitory shunting, and calcium-based online learning. These are not biological luxuries; they are computational capabilities (coincidence detection, online STDP, top-down inhibitory control) that make L1 more powerful than a LIF array.

The D compartment circuit—10 μF, 1 kΩ, a few weight resistors—is the first physical piece. It is buildable today on a breadboard, and has not yet been built; the measurement bench is written and awaiting the parts. The progression from there to the full DSTC chip is specified: S compartment (Schmitt trigger), θ compartment (slow RC), calcium compartment (slow RC on spike output), MOSFET NMDA gate. Each step is a standard analog component.

The Python code (`dst_model.py`, `dstc_model.py`, `D_compartment.py`) is not a simulation to be discarded—it is the specification. Every equation in the code corresponds directly

to a circuit element. The software and hardware are the same system at different levels of abstraction.

SPECTRUM needs six layers. Three now have concrete instantiations: DSTC at L1, analog at L4, ARBOR at L6. The integration is the research agenda.

References

[1] Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS 2017*.

[2] Hodgkin, A.L., & Huxley, A.F. (1952). A quantitative description of membrane current and its application to conduction and excitation in nerve. *Journal of Physiology*, 117(4), 500–544.

[3] Izhikevich, E.M. (2003). Simple model of spiking neurons. *IEEE Transactions on Neural Networks*, 14(6), 1569–1572.

[4] Izhikevich, E.M. (2004). Which model to use for cortical spiking neurons? *IEEE Transactions on Neural Networks*, 15(5), 1063–1070.

[5] Brette, R., & Gerstner, W. (2005). Adaptive exponential integrate-and-fire model as an effective description of neuronal activity. *Journal of Neurophysiology*, 94(5), 3637–3642.

[6] Lapicque, L. (1907). Recherches quantitatives sur l'excitation électrique des nerfs traitée comme une polarisation. *J. Physiol. Pathol. Gen.*, 9, 620–635.

[7] Davies, M., et al. (2021). Advancing neuromorphic computing with Loihi. *Proceedings of the IEEE*, 109(5), 911–934.

[8] IBM Research (2023). NorthPole: An architecture for neural network inference with a 12nm chip. *Science*, 382(6668).

[9] Mainen, Z.F., & Sejnowski, T.J. (1996). Influence of dendritic structure on firing pattern in model neocortical neurons. *Nature*, 382, 363–366.

[10] Bi, G., & Poo, M. (1998). Synaptic modifications in cultured hippocampal neurons. *Journal of Neuroscience*, 18(24), 10464–10472.

[11] Shouval, H.Z., et al. (2002). A unified model of NMDA receptor-dependent bidirectional synaptic plasticity. *PNAS*, 99(16), 10831–10836.

[12] Lin, W.Z. (2026). ARBOR: Adaptive Routing with Branch-Organized Reasoning. *arXiv preprint*.

[13] Lin, W.Z. (2026). FLUID: A self-structuring architecture for open-ended artificial intelligence. *arXiv preprint*.

[14] Lin, W.Z. (2026). SPECTRUM: A Multi-Substrate Hybrid Architecture for General-Purpose Artificial Intelligence. *arXiv preprint*.